

FRC Scouting App — Project Handoff Summary (v8)

A complete, self-contained record of the offline FRC scouting desktop application build: goals, source material, architecture, what has been implemented and verified in every stage through Stage 14, and exactly what a future session needs to know to continue with no access to any original chat history.

This document supersedes the earlier v7 handoff PDF — it contains everything that PDF did, brought up to date, plus one new stage delivered in a follow-up session: Stage 14, a new "Match Simulator" tab that lets the user type in any six team numbers and see the app's win-odds/expected-score output for that hypothetical matchup, whether or not it exists on the imported Match Schedule. See Section 19 for the full write-up. If this document and the code ever disagree, treat the code as ground truth and update this document.

1. Project Goal

Ian is building a fully offline Python desktop application that replicates the logic of a complex FRC (FIRST Robotics Competition) scouting Google Spreadsheet as a standalone tool. Requirements:

- 100% offline at runtime — no network calls, no external API, no cloud database.
- Runs on both Windows and Mac using only standard, freely-licensed components.
- Local storage only (SQLite file on disk).
- Four required UI tabs, in this order: Match Lookup, Team Lookup, Pick List, Rankings — matching the first four tabs of the source Google Sheet, whose content must appear on the matching tabs. (Stage 14 added a fifth, optional tab, Match Simulator, that is NOT part of this original four-tab requirement — see Section 3 and Section 19.)
- The remaining ~28 sheets in the source spreadsheet contain the backend calculation logic and must be replicated exactly, not approximated casually — the user has explicitly and repeatedly stressed correctness over speed ("make no mistakes", "it is imperative this code is also correctly implemented").
- The scoring/forecasting logic implements a custom, non-Monte-Carlo skewed-distribution statistical method for forecasting robot performance and predicting match outcomes, documented in a PDF titled "Scouting Algorithm Overview" (a.k.a. Scouting_Algorithm_Overview_4.pdf).

2. Source Material

2.1 The Google Sheet

Original file: "Copy of Worlds Scouting Sheet" (Google Sheets ID 1RcHI_oUH3XHg3i4FAZgDR5eB0Wuq5HtgWGzZlrreO7c). It contains 32 tabs and roughly 215,687 individual formula/value cells. The first four tabs share the exact names of the four required UI tabs; the other 28 tabs are backend calculation sheets.

2.2 How the formulas were extracted

Google Sheets formulas do not survive a plain .xlsx export reliably (Google-only functions like QUERY, ARRAYFORMULA, IMPORTRANGE, FILTER, XLOOKUP, VSTACK can be flattened or dropped). To get every formula as durable plain text, an Apps Script was written and run directly inside the spreadsheet (Extensions → Apps Script) that:

- Iterates every sheet and every used cell, recording Sheet name, Cell (A1 notation), Value (display value), and Formula (raw formula text, or blank for non-formula cells) into a new tab called FORMULA_EXPORT.

- Is resumable — because Apps Script has a ~6-minute execution limit, and this workbook has 215k+ cells, the script checkpoints its progress (current sheet index, row index, output row) in Script Properties every 500 rows and on a 5m20s time budget, so re-running the same function (exportAllFormulas) continues exactly where it left off. Helper functions resetExport() and checkExportStatus() were also provided.

The FORMULA_EXPORT tab was then downloaded as CSV: Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv (215,687 data rows, columns: Sheet, Cell, Value, Formula). This CSV is the ground-truth source for every formula translated into Python in this project, and the ground-truth source used to find and verify every bug described in Sections 9, 10, 11, and 14 below. Keep this CSV — it is required to verify or extend any calculation.

2.3 The algorithm PDF

"Scouting Algorithm Overview" (Scouting_Algorithm_Overview_4.pdf, 17 pages) describes the underlying statistical method in prose/derivation form: fitting a skew-normal-like distribution to each team's historical scores WITHOUT Monte Carlo simulation, by (1) picking a transform (sqrt / log10 / reciprocal, or their power-inverses for left-skewed data) that best symmetrizes a team's score history, (2) fitting a normal PDF over the transformed axis using the transformed data's own mean/stddev, (3) scaling the resulting curve so its peak is 1, with a hard cutoff around the 90th–98th percentile and a broadening adjustment for weak teams, and (4) combining three teammates' curves and comparing two alliances' combined curves to estimate win probability. Equation OCR in a couple of spots (e.g. the sigma_adj approximation) was ambiguous (missing radicals/exponents); ground truth was resolved by cross-checking against the exported formulas (2.2) instead of trusting the OCR text alone.

2.4 A note on a missing sheet

Several formulas in the Team Lookup sheet (team name, weak-to-defense notes, climb-confidence flags) reference a sheet called "Super Scouting" that does not exist anywhere in the 32-sheet export. The user confirmed this sheet is not needed and instructed it be ignored — those specific Team Lookup fields are intentionally left unimplemented. Unchanged since the original build, and unchanged through Stage 14.

3. Delivery Approach: Staged, Not One-Shot

Given the size (32 sheets, 215k+ cells) and the presence of genuinely novel statistical logic (not boilerplate spreadsheet math), the build was explicitly staged rather than attempted in one pass, to avoid silently getting the core algorithm wrong:

- Stage 1 — the raw data pipeline (10 sheets). Complete and delivered.
- Stage 2 — the skew-normal curve fitting + defense/compatibility stats (4 more sheets). Complete and delivered.
- Stage 3 — power rankings, alliance combination, match odds, and the real Pick List / Rankings logic (remaining sheets). Complete and delivered, with one explicitly flagged area needing the user's validation (see Section 6.5).
- Stage 4 — Data Import Redesign: replaced the original three-CSV granular input model with a streamlined two-CSV model built around the user's real data format, fixed a zero-collapse bug in the Adjusted Score calculation, and validated the full pipeline end-to-end against the user's actual competition data for the first time. Full detail in Section 21.
- Stage 5 — Adjusted Score Variance Bug Investigation: three real bugs found and fixed in the import/calculation chain between Score Variance and Adjusted Score Variance. Full detail in Section 9.
- Stage 6 — Experiment: at the user's explicit request, Adjusted Score Variance was removed from the calculation pipeline entirely and replaced everywhere with plain Score Variance. Full detail in Section 10.

- Stage 7 — Expected Score Bug Investigation: root-caused and fixed two distinct, real bugs in `calculations/alliance_generator.py`. Full detail in Section 11.
- Stage 8 — UI Quality-of-Life Improvements: dropdown selectors replaced with type-in number fields on Match Lookup and Team Lookup, and a "Sort by" dropdown added to Rankings. Full detail in Section 12; validation detail in Section 13.
- Stage 9 — Bug Fix: New-Competition Import Was Mixing With Old Competition Data. Fixed with an explicit per-import "Update current competition" vs. "New competition" choice. Full detail in Section 14.
- Stage 10 — Team Lookup: Score Trend Chart. Full detail in Section 15.
- Stage 11 — Team Lookup: Panel Cleanup. Full detail in Section 16.
- Stage 12 — Match Lookup: Score Distribution Chart. Full detail in Section 17.
- Stage 13 — Startup Performance Optimization: removed a single expensive `scipy.stats.norm.pdf()` call site from the curve-fit, removed two unused pip dependencies (numpy, pandas), and made matplotlib's import lazy in both chart-drawing tabs. Full detail in Section 18.
- Stage 14 — Match Simulator Tab: added a brand-new, fifth UI tab that lets the user type in any six team numbers (not necessarily on the schedule together, or on the schedule at all) and see the same win-odds/expected-score/score-distribution output Match Lookup shows for a real match, minus the real-world recorded score (which doesn't exist for a hypothetical matchup). The alliance-panel and chart-drawing code Match Lookup already had was factored out into a new shared module so the two tabs can never visually drift apart. Full detail in Section 19.

4. Current File Structure

Delivered as a project folder `frc_scouting_app/` (most recently shared zipped as `frc_scouting_app_16.zip` — the zip shared at the end of the Stage 14 session is the current, authoritative version; if a newer-numbered zip has been shared since, that one is authoritative instead). `frc_scouting_app_11.zip` was the Stage 9 checkpoint; `_12` added Stage 10's chart, `_13` added Stage 11's panel cleanup, `_14` added Stage 12's chart, `_15` added Stage 13's startup-performance changes, and `_16` added Stage 14's Match Simulator tab (a new `ui/match_simulator_tab.py` and `ui/alliance_display.py`, plus small edits to `ui/match_lookup_tab.py` and `ui/app.py`) — each is superseded by the next, with only that stage's changes as its diff from the previous zip. This is the full, current layout:

- `main.py` — entry point, run this. Unaffected by Stage 14.
- `database.py` — SQLite schema (local file, no server). Unaffected by Stages 10–14.
- `data_import.py` — CSV importers for the 2 active input tables (`manual_raw_data`, `match_schedule`). Unaffected by Stages 10–14.
- `requirements.txt` — matplotlib only (the project's sole pip dependency since Stage 13). Unaffected by Stage 14 — the Match Simulator tab reuses matplotlib exactly as Match Lookup already did, so no new dependency was needed.
- `README.md` — setup + full data-import format reference + every stage's writeup, now including a Stage 13 section (present in this PDF since v7 but missing from `README.md` until this session — added during the Stage 14 session so the two stay in sync) and a new Stage 14 section.
- `calculations/` — completely unaffected by Stage 14, in every file, with no exceptions. Every function this tab calls (`build_alliance_distribution()`, `compute_match_odds()`) already existed and is called exactly as Match Lookup calls it — see Section 19.4.

- `ui/app.py` — main window, menu, notebook of tabs. Changed in Stage 14: imports and instantiates the new `MatchSimulatorTab`, adds it to the notebook as a 5th tab (after Rankings) labeled "Match Simulator", and calls its `refresh()` method alongside the other four tabs' `refresh()` calls inside `recalculate()`. No other change to this file in Stage 14 — the Stage 9 `ImportModeDialog` and `_import()` logic are untouched.
- `ui/match_lookup_tab.py` — tab 1: Match Lookup (live win odds). Changed in Stage 14: the alliance-panel-building code (previously a local `alliance_panel()` closure inside `_render()`) and the entire `_draw_alliance_chart()` method were moved out of this file into the new `ui/alliance_display.py` module (`build_alliance_panel()` and `draw_alliance_chart()` respectively); `_render()` now calls those two shared functions instead of local ones. This was a pure code-motion — no calculation, layout, or visual output changed. See Section 19.4 for why this refactor was done and how it was verified to be behavior-preserving.
- `ui/alliance_display.py` — NEW in Stage 14. A small shared module with no calculation logic of its own — `build_alliance_panel()` renders one alliance's Treeview of teams plus its actual/expected score and win-probability labels, and `draw_alliance_chart()` draws the smoothed red/blue score-distribution overlay chart (the exact same smoothing logic Stage 12 introduced, unchanged). Both `ui/match_lookup_tab.py` and the new `ui/match_simulator_tab.py` call into this module so their displays can never drift apart. See Section 19.
- `ui/match_simulator_tab.py` — NEW in Stage 14. Tab 5: Match Simulator. Six digit-only Entry fields (three red, three blue) plus a "Simulate" button; calls the same calculations.`alliance_generator.build_alliance_distribution()` and `calculations.match_odds.compute_match_odds()` Match Lookup calls, then renders the result via `ui/alliance_display.py`'s shared functions with `actual_score=None` (no real-world score exists for a hypothetical match). See Section 19 for full detail.
- `ui/team_lookup_tab.py` — tab 2: Team Lookup. Unaffected by Stage 14.
- `ui/pick_list_tab.py` — tab 3: Pick List. Unaffected by Stage 14.
- `ui/rankings_tab.py` — tab 4: Rankings. Unaffected by Stage 14.
- `data/scouting.db` — created automatically on first run

All 32 source sheets have a corresponding Python implementation. Every `.py` file's docstring documents which sheet(s) it recreates and the exact formula it is based on — treat those docstrings as the authoritative, line-by-line spec. `calculations/alliance_generator.py`'s docstring remains the single best place to start if you need to touch alliance-combination logic again. Stage 14 added one new module (`ui/alliance_display.py`) with its own module-level docstring explaining its shared-rendering purpose, updated `ui/match_lookup_tab.py`'s module docstring to mention the refactor, and added a new module docstring in `ui/match_simulator_tab.py` — it did not change any docstring in `calculations/`.

5. Runtime / Environment

- Language: Python 3.11+ (standard installer from python.org on both platforms).
- GUI: tkinter (ships with the standard Python installer — on Mac, must be the official python.org installer, not a Tk-less Homebrew build).
- Local data store: SQLite via the built-in `sqlite3` module — a single file at `data/scouting.db`.

- Math/visualization dependencies: matplotlib only, imported lazily (as of Stage 13) inside the chart-drawing functions that need it. As of Stage 14, there is exactly one such function shared by two tabs: `ui/alliance_display.py::draw_alliance_chart()`, called from both `ui/match_lookup_tab.py` and `ui/match_simulator_tab.py`'s `_render()/_simulate()` methods (`ui/team_lookup_tab.py` still has its own separate `_draw_score_chart()` for the unrelated Score Trend chart). `numpy`, `scipy`, and `pandas` remain removed as of Stage 13 and were not reintroduced by Stage 14 — matplotlib is still the project's only pip dependency.
- Zero network calls anywhere in the codebase — confirmed by design, no HTTP/socket libraries are imported. Still true as of Stage 14.
- Setup: `pip install -r requirements.txt` then `python main.py`.

Stage 14 added no new dependencies. It reuses the exact matplotlib/TkAgg embedding pattern (Figure + FigureCanvasTkAgg, imported lazily inside the drawing function) that Stages 10 and 12 already established.

6. What Has Been Implemented (Stage-by-Stage Detail)

Section 6's contents (6.1 through 6.6 — the local data model, Stage 1's raw data pipeline, Stage 2's curve fitting and defense/compatibility calculations, Stage 3's power rankings/pick list/alliance odds, the known Blue/Red cross-labeling uncertainty, and the other explicit assumptions/configurable constants) are entirely unaffected by Stage 14 and are unchanged from the v7 PDF. Stage 14 added no new calculation, changed no existing calculation, and reads every value it displays from `calculations.pipeline.ComputedData` exactly as `ui/match_lookup_tab.py` already did. Refer to the v7 PDF's Section 6 (reproduced in full in this handoff's companion materials / prior sessions) for the complete calculation-by-calculation writeup; nothing there needs to change. The one addition worth restating here explicitly: `calculations/alliance_generator.py::build_alliance_distribution()` and `calculations/match_odds.py::compute_match_odds()` (Section 6.4) are now called from two call sites instead of one (`ui/match_lookup_tab.py` and `ui/match_simulator_tab.py`), but the functions themselves, and everything they compute, are completely unchanged.

7. Current UI State (all 5 tabs are fully wired, no placeholders)

- Match Lookup — type a match number into the "Match #:" field (digits only) and press Enter, click "Go", or click away from the field. Shows both alliances' three teams with Score Variance avg/RRP, the actual recorded score, and (built live, not cached) expected score + win probability, plus the Stage 12 Score Distribution chart. Unaffected in appearance or behavior by Stage 14 — only the code location of its panel/chart-drawing logic changed (Section 19.4).
- Team Lookup — unaffected by Stage 14. See the v7 PDF's Section 7 for full detail (Score Trend chart, the four remaining stat panels after Stage 11's cleanup).
- Pick List — unaffected by Stage 14.
- Rankings — unaffected by Stage 14.

- Match Simulator — NEW in Stage 14. Six digit-only Entry fields, labeled "Red Alliance:" (three boxes) and "Blue Alliance:" (three boxes), plus a "Simulate" button (Enter in any box also triggers it). All six boxes must be filled before Simulate does anything — an empty submission shows "Enter all six team numbers above, then click Simulate." Typing the same team number into more than one box shows "Each team number can only appear once in a match..." instead of running the calculation. On a valid submission, shows the same two alliance panels (team list with Score Variance avg/RRP, computed expected score, win probability) and Score Distribution chart Match Lookup shows for a real match — but with no "Actual recorded score" line, since a hypothetical matchup has no real-world score. If one or more of the six teams has no fitted Normalized Distribution (e.g. too little scouted data), shows "Win odds unavailable: Missing fitted distribution for team(s): [...]" exactly as Match Lookup does for a real match with the same problem.
- Data menu — unaffected by Stage 14; Import Match Schedule / Import Raw Data / Set My Team Number / Recalculate Now, each exactly as described in the v7 PDF's Section 7 and Section 14.

8. Testing Performed So Far

Everything through Stage 13 (headless synthetic-data smoke tests after every stage; Stage 3's range/tier/probability-sum checks; Stage 4's first real-data validation; Stage 5's cell-by-cell FORMULA_EXPORT cross-check; Stage 6's before/after Goal Ally RPP comparison; Stage 7's 110-match mean-absolute-error and win-direction-accuracy comparison; Stage 8's ast.parse syntax check; Stage 9's bug-reproduction-then-fix-confirmation test; Stage 10's headless Agg-backend PNG render plus the user's own real-machine confirmation; Stage 11's ast.parse check plus the user's own real-machine confirmation; Stage 12's 110-match smoothing stress test plus three spot-checked PNG renders, not yet confirmed on a real machine at the time; Stage 13's before/after numeric diff of build_all()'s full output, showing floating-point-noise-level differences at worst and bit-for-bit identical Pick List/RPP output, plus import-timing measurements) is unaffected by Stage 14 and unchanged from the v7 PDF's Section 8. See that section (or the corresponding stage's own section in this PDF — Sections 9 through 18) for full detail on each.

Stage 14 testing, performed in this session:

- Every .py file in the project (including the two new/changed ui/ files) was re-syntax-checked with Python's `ast.parse()` after the edits and all parsed cleanly.
- The refactor of ui/match_lookup_tab.py (moving its panel/chart-drawing code into ui/alliance_display.py) was reviewed line-by-line against the pre-refactor version to confirm it is a pure code-motion: the same Treeview columns, the same label text and formatting strings, the same smoothing window/stride constants, and the same empty-state placeholder text are all preserved verbatim in the new shared module.
- The pipeline was re-run headlessly against the user's real, uploaded CSVs (Untitled_spreadsheet_-_Match_Schedule (1).csv and Untitled_spreadsheet_-_Raw_Data (1).csv — a different competition's data than the CSVs referenced in Sections 9–18, which had a 75-team roster; this dataset produced a 45-team roster from a 90-match schedule and 538 imported raw-data rows, 26 schedule_conflicts, 0 likely_typo_corrections) via `data_import.import_match_schedule_csv()` / `import_raw_data_csv()` and `calculations.pipeline.build_all()`, confirming the import and full calculation pipeline still run cleanly end-to-end (unsurprising, since Stage 14 touched no file under calculations/, but confirmed directly rather than assumed).

- `calculations.alliance_generator.build_alliance_distribution()` and `calculations.match_odds.compute_match_odds()` — the same two functions `ui/match_simulator_tab.py` calls — were then called directly, headlessly, on an arbitrary six-team combination that does NOT appear together on that competition's real schedule (teams 127/148/231 vs. 418/457/2158, picked only because they were the first six team numbers with a fitted distribution, with no attempt to hand-pick a favorable result): this produced a red win probability of 40.9%, a blue win probability of 59.1% (summing to exactly 1.0, as required), and expected scores of approximately 281 (red) and 307 (blue) — sane, well-formed output for a hypothetical matchup, confirming the simulator's core logic path works correctly end-to-end.
- The Tkinter GUI itself was NOT visually smoke-tested in the build environment used for this session — no display is available there, and (as of this session) tkinter is still not installed in that environment at all, the same standing limitation noted in Stages 8, 9, 12, and 13. This is the top-priority open item going into the next session — see Section 20.

9. Stage 5 — Adjusted Score Variance Bug Investigation & Fixes

Unaffected by Stage 14; unchanged from the v7 PDF. Symptom: Adjusted Score Variance, as calculated by the app, differed from what the live Google Sheet calculated for the same data. Three bugs were found and fixed by importing the user's real CSVs through the actual production import functions and comparing every intermediate value cell-by-cell against `Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv`'s own cached values:

- Bug 1 — Match Schedule was not treated as authoritative during Raw Data import; `import_raw_data_csv()` now always tries a `match_schedule` lookup for a row's team+match first, falling back to the row's own `Alliance/Position` cell only if that lookup fails.
- Bug 2 — Score Variance was not capped to each team's first 15 chronological matches, which the source sheet's own layout (columns B:P) requires; `build_score_variance()` now truncates accordingly.
- Bug 3 — Raw Data Processing's real lookup is occurrence-indexed (a `COUNTIF`-then-`XLOOKUP` pattern counting how many times a team has appeared anywhere in the schedule up to that row, then taking that Nth value from the team's Score Variance history), not a direct (team, match) lookup — confirmed as a genuine, reproducible quirk of the source sheet itself and replicated exactly in `build_raw_data_processing()`.
- A fourth issue, found via user feedback after Bug 1 shipped: an evidence-based typo-correction feature was added to `import_raw_data_csv()` that auto-corrects a Match Schedule slot only when the team currently on record is a total orphan (appears exactly once, zero corroborating raw-data rows) while the conflicting row's team is well-established (3+ other rows). Every correction is reported via `likely_typo_corrections`, never silent; every more ambiguous conflict is left unresolved-but-reported via `schedule_conflicts`.

Validation on the user's real (original) CSVs: 625 Raw Data rows imported, 29 skipped, 0 unresolved-alliance rows, 34 `schedule_conflicts`, 1 `likely_typo_corrections` (confirmed correct by the user). Score Variance matched 624/625 (99.8%) against `FORMULA_EXPORT`; Adjusted Score Variance matched 561/594 (94.4%) once two confirmed non-code-bug causes were excluded.

10. Stage 6 — Experiment: Adjusted Score Variance Removed

Unaffected by Stage 14; unchanged from the v7 PDF. At the user's explicit request, `calculations/pipeline.py::build_all()` no longer calls `score_variance.build_adjusted_score_variance()` — every downstream function that used to receive Adjusted Score Variance now receives plain Score Variance instead. This is current, active, production behavior; the user reviewed the results and decided to keep it. `build_adjusted_score_variance()` itself was not deleted and remains fully callable/testable in isolation; Section 10.4 of the v7 PDF documents the exact, still-valid revert path (which now needs one small adjustment: Team Lookup's plain "Score Variance" panel this revert path

used to point at no longer exists as of Stage 11 — see that section for detail). A concrete data point: Goal Ally RPP moved from ≈ 0.836 (Adjusted Score Variance active) to ≈ 0.780 (plain Score Variance) on the original competition's real data, with team count (75) and Pick List size (74) unchanged.

11. Stage 7 — Expected Score Bug Investigation & Fixes

Unaffected by Stage 14; unchanged from the v7 PDF. The user reported Match Lookup's expected score was far below real match scores, "as if the alliance combination were only picking up one of the three teammates' scores." Two real bugs were found in `calculations/alliance_generator.py::build_alliance_distribution()` by tracing FORMULA_EXPORT's "Alliance Base Generator", "Blue Alliance Generator", and "Match Odds" sheets cell-by-cell:

- Bug 1 — the ascending branch of each teammate's split curve was combined in the wrong point order; the sheet's own `SORT(...,FALSE())` formula reverses it, which `_split_branches()` now replicates.
- Bug 2 — the alliance-level combined curve was capped at the same 1–500 axis used for a single robot, even though a 3-robot combined score routinely exceeds that (confirmed from both FORMULA_EXPORT's own cached values and the user's real recorded scores, 35% of which exceed 500). Fixed by giving the alliance-level curve its own wider axis, `ALLIANCE_X_AXIS_MAX = 3 * X_AXIS_MAX (1500)` — this constant is exactly what Stage 14's Match Simulator inherits automatically, since it calls the same `build_alliance_distribution()` function.

Validation: mean absolute error between expected and actual alliance score dropped from ≈ 175 to ≈ 100 points across all 110 real matches with a recorded score; win-direction accuracy rose from 67% (74/110) to 69% (76/110). Bug 2 is confirmed present in the original Google Sheet itself, not just this port — this app's fix is a deliberate, intentional divergence made because the un-fixed behavior was measurably less accurate.

12. Stage 8 — UI Quality-of-Life Improvements

Unaffected by Stage 14; unchanged from the v7 PDF. Match Lookup and Team Lookup's read-only dropdowns were replaced with digit-validated `ttk.Entry` fields (Enter/FocusOut/a "Go" button all submit) — this is the same digits-only Entry pattern Stage 14 reused for the Match Simulator's six team-number fields, right down to the `self.register(self._validate_digits) / validate="key"` wiring. Rankings gained a "Sort by" dropdown (RPP/ROP/RDP/Distance). No calculation code was touched.

13. Google Sheet Validation — First Pass Complete, Bookmarked for Follow-Up

Unaffected by Stage 14; unchanged from the v7 PDF. During the Stage 8 session, the user compared this app's Match Lookup output against the live Google Sheet's own Match Odds for at least one real match; the two produced different (not identical) percentages, which the user judged expected and "passable for now" given it is "impossible to have a perfect port" of a 215,000-cell workbook. No specific match number or percentages were captured, so none should be reported without a fresh, logged comparison. Two already-understood sources of expected divergence: the Stage 7 Bug 2 axis-truncation fix (this app's number is the more accurate one there) and the still-unconfirmed Blue/Red cross-labeling question (Section 6.5 of the v7 PDF). This remains open and untouched by Stage 14 — see Section 20 for how to make progress on it.

14. Stage 9 — Bug Fix: New-Competition CSV Import Mixing With Old Competition's Data

Unaffected by Stage 14; unchanged from the v7 PDF. The user found that importing a second competition's CSVs on top of an already-loaded first competition's data silently mixed the two together rather than replacing it — `match_schedule`'s primary-key upsert interleaved two competitions' same-numbered matches, and `manual_raw_data` (no natural key) simply doubled on every re-import. Root cause: `ui/app.py::_import()` never used the `replace_existing` parameter both importer functions already supported. Fixed with a new `ImportModeDialog` (Section 4 of the v7 PDF) offering "Update current competition" (merge, `replace_existing=False`) vs. "New competition" (wipes both tables via the new `database.clear_active_competition_data()`, then imports with `replace_existing=True`). This dialog and its wiring are completely unaffected by Stage 14 — the Match Simulator tab does not import any data of its own; it only reads whatever competition's data is currently loaded via the existing Data menu.

15. Stage 10 — Team Lookup: Score Trend Chart

Unaffected by Stage 14; unchanged from the v7 PDF. A "Score Trend" line chart was added to the top of Team Lookup, plotting `self.data.score_variance[team].values` (the same per-team, ≤ 15 -match chronological list Stage 5 verified) via matplotlib's TkAgg backend, embedded through `ui/team_lookup_tab.py::_draw_score_chart()` — a separate function from the one Stage 14's shared module introduced, since it draws a different kind of chart (a single team's history line, not two alliances' overlaid distributions) for a different tab. matplotlib was added as the project's first new dependency in this stage.

16. Stage 11 — Team Lookup: Panel Cleanup

Unaffected by Stage 14; unchanged from the v7 PDF. After the Stage 10 chart made Team Lookup too tall to fit on screen, three redundant/unused stat panels (Score Variance, Skew Adjusted Spread, Defense) were removed at the user's request, and the remaining four panels reflowed into a 3-row/4-panel grid. The user confirmed on their own machine this alone fixed the fit problem, so no scrolling feature was ever built.

17. Stage 12 — Match Lookup: Score Distribution Chart

Unaffected in substance by Stage 14, though its implementation moved files — see Section 19.4. A chart was added below Match Lookup's red/blue alliance panels, overlaying both alliances' combined-score curves (`AllianceDistribution.empirical_curve`) on one plot. Because that curve is built via an exact-match lookup onto a 1500-point integer axis (mirroring the source sheet's own VLOOKUP behavior), most integer scores land on density exactly 0 next to points near 1.0 — real, not a bug, but unreadable if plotted raw. The chart applies a display-only moving average (window 25) and plots every 6th point, independently rescaling each curve so its own peak is 1.0; this smoothing never touches `red_dist/blue_dist` or any calculation — every number in the panels above the chart still comes from the raw, unsmoothed curve. Validated against all 110 real matches (all produced a valid smoothed curve) and three spot-checked matches across the win-probability range, rendered headlessly via matplotlib's Agg backend. As of the v7 PDF this chart had not yet been confirmed on the user's real machine; as of this v8 PDF, the user reported at the start of this session that "the latest results look good," which — together with the earlier real-machine confirmations of Stages 10 and 11 — is treated as confirmation that Stage 12's chart (and Stage 13's startup changes, both delivered in the same prior session) are working correctly on the user's actual machine. See Section 19.4 for the Stage 14 refactor that relocated this chart's drawing code (unchanged in behavior) into `ui/alliance_display.py::draw_alliance_chart()`.

18. Stage 13 — Startup Performance Optimization

Unaffected by Stage 14; unchanged from the v7 PDF. `calculations/distributions.py::build_skew_distributions()` was calling `scipy.stats.norm.pdf()` roughly 37,500 times for a 75-team roster; a full-project grep confirmed numpy and

pandas (both listed in requirements.txt since the original build) were never actually imported anywhere. Fix: `scipy.stats.norm.pdf()` was replaced with a private `_normal_pdf(x, loc, scale)` function (~10 lines, standard-library math module only), and numpy/scipy/pandas were removed from requirements.txt entirely, leaving matplotlib as the sole dependency. matplotlib's own imports in `ui/match_lookup_tab.py` and `ui/team_lookup_tab.py` were moved from module scope into their respective chart-drawing methods, so a fresh install with nothing imported yet doesn't pay that cost at startup either.

Validation: `build_all()`'s full output was diffed before/after against the user's real CSVs — `skew_distribution_curves` matched to floating-point noise (max difference $\approx 1.1\text{e-}13$), `normalized_distributions` similarly (max difference $\approx 4.4\text{e-}16$, with `mode_x/upper_cutoff` bit-for-bit identical), and `pick_list/rpp` were bit-for-bit identical. Timing: `build_all()` alone dropped from $\approx 0.63\text{s}$ to $\approx 0.02\text{s}$. As of the v7 PDF this had not been confirmed on a real machine; as of this v8 PDF the user's "latest results look good" confirmation at the start of this session covers this stage too (see Section 17 above) — the app opening noticeably faster and both charts still rendering correctly are treated as confirmed, though no specific stopwatch timing or screenshot was captured from the user's own machine, so if a future session wants a precise real-machine timing number for the record, that would need to be captured fresh.

The one item from Stage 13 that remains genuinely open and unaffected by anything in this session: `ui/app.py::ScoutingApp.recalculate()` still eagerly calls `.refresh()` on all tabs (now five, including the new Match Simulator) at startup, before the user can see any but the first. Section 18.8 of the v7 PDF flagged deferring non-visible tabs' `refresh()` until first shown as a good, not-yet-implemented follow-up optimization; Stage 14 made this marginally more relevant (one more tab is now eagerly refreshed at startup) but did not address it — see Section 20.

19. Stage 14 — Match Simulator Tab

19.1 Request

The user asked for a new tab that behaves like Match Lookup, but instead of typing in a match number, lets the user type in each of the six teams for a hypothetical match directly (three red, three blue), then click a "Simulate" button to see everything Match Lookup normally shows — except the real-life match score, since a hypothetical match has none. The user still wanted the app's computed guess at the match score (i.e. the expected score) to be shown. The stated goal was to let the user plug in different hypothetical matchups and see how they would play out — for example, trying out an alliance combination that isn't actually on the schedule.

19.2 Design decisions

Placement: added as a fifth tab, named "Match Simulator", placed after Rankings (i.e. last). This was a deliberate choice to avoid disturbing the existing, documented invariant (Section 1 and elsewhere throughout this document) that the first four tabs correspond one-to-one with the source Google Sheet's first four tabs — Match Simulator has no corresponding sheet in the source spreadsheet at all; it is a new capability built on top of the existing algorithm, not a port of anything.

What "everything Match Lookup shows, except the real score" means concretely: the two alliance panels (each team's Score Variance average/RRP, the computed expected score, and the win probability) and the Score Distribution chart are shown exactly as on Match Lookup. Only the "Actual recorded score: N" line is omitted — not shown as 0, not shown as a dash, simply not present, since printing any number there could be mistaken for a real result. The expected score (the app's "guess") and win probability ARE shown, unconditionally, whenever the six teams' fitted distributions are available — this is the actual point of the tab per the user's request.

Input validation: the six team-number fields use the exact same digits-only `validate="key"` Entry pattern already established on Match Lookup and Team Lookup (Section 12 / Stage 8) for consistency and to avoid introducing a new input-handling pattern. All six fields must be non-empty before Simulate does anything (an

incomplete submission shows a prompt rather than erroring). If the same team number is typed into more than one of the six boxes, the tab refuses to run the calculation and shows an error instead — a real FRC alliance can never include the same robot twice, so this guards against a nonsensical simulated match (e.g. a "three of the same robot" alliance, which would otherwise silently produce a technically-valid but meaningless result from `build_alliance_distribution()`).

Missing-data handling: if one or more of the six typed teams has no entry in `self.data.normalized_distributions` (e.g. a team number that doesn't exist in the imported roster, or one with too little scouted data to fit a curve), the tab shows "Win odds unavailable: Missing fitted distribution for team(s): [...]" — the exact same message format Match Lookup already used for the same underlying condition, so the two tabs behave identically when data is missing.

19.3 Why the panel/chart-drawing code was refactored into a shared module

Before this stage, the Treeview-based alliance panel and the smoothed score-distribution chart were built by code living directly inside `ui/match_lookup_tab.py`: a local `alliance_panel()` closure defined inside `_render()`, and a `_draw_alliance_chart()` method on the `MatchLookupTab` class. Simply copying that code into the new `ui/match_simulator_tab.py` was considered and rejected: any future change to how a panel or chart looks (a color tweak, a new stat added to the Treeview, a change to the smoothing window) would then need to be made twice, and a session that only remembered to update one of the two tabs would leave them silently inconsistent — exactly the kind of silent-drift failure mode this project's own stated principles (see prior sessions' guidance: "faithfulness over elegance," "silent data corruption is the worst failure mode") warn against, generalized here from calculation drift to display drift.

Instead, the panel-building code became `build_alliance_panel(parent, title, teams, score_variance, *, actual_score=None, win_prob=None, expected_score=None)` and the chart-drawing code became `draw_alliance_chart(parent, red_dist, blue_dist)`, both moved into a new module, `ui/alliance_display.py`, with no calculation logic of their own — they only format and draw values the caller already computed. `ui/match_lookup_tab.py::_render()` was edited to call these two shared functions instead of its own local versions (passing `actual_score=(m["red_score"] or 0)`, i.e. never `None`, exactly preserving its pre-refactor behavior of always showing an actual-score line, even a "0" one, for a real scheduled match). `ui/match_simulator_tab.py::_simulate()` calls the same two functions with `actual_score=None`, which is what makes that one line disappear for a hypothetical match. Both tabs now share exactly one implementation of "what an alliance panel looks like" and "how the score-distribution chart is drawn."

19.4 What did NOT change

No file under `calculations/` was touched, added, or removed. `calculations.alliance_generator.build_alliance_distribution()` and `calculations.match_odds.compute_match_odds()` are called by `ui/match_simulator_tab.py` exactly as `ui/match_lookup_tab.py` already called them — same function signatures, same arguments (a 3-tuple of `NormalizedDistribution` objects per alliance), same return types. The refactor described in 19.3 is a pure code-motion for Match Lookup: the Treeview's column headers/widths, the exact label text and number-formatting strings ("Actual recorded score: {actual_score:g}", "Expected score: {expected_score:.0f}", "Win probability: {win_prob*100:.1f}%"), and the chart's smoothing window (25), stride (6), colors, and empty-state placeholder text ("Score distribution unavailable for this match.") are all preserved verbatim in `ui/alliance_display.py` — Match Lookup's on-screen appearance and behavior are unchanged by this stage.

19.5 Validation results

See Section 8's "Stage 14 testing" entry for the full detail (ast.parse syntax check across the whole project; a line-by-line review confirming the refactor is behavior-preserving; a fresh headless pipeline run against the user's real, uploaded CSVs for a different competition than earlier stages' — 45 teams, 90 scheduled matches, 538 raw-data rows imported; and a direct, headless call to `build_alliance_distribution()/compute_match_odds()` on an arbitrary six-team combination not found together on that schedule, producing a well-formed 40.9%/59.1% split with expected scores of $\approx 281/\approx 307$, win probabilities summing to exactly 1.0). As with Stages 8, 9, 12, and 13, the Tkinter GUI itself has not been visually smoke-tested in the build environment used for this work — no display is available there. This is the top-priority item for the next session; see Section 20.

19.6 Code changes summary (Stage 14)

`ui/alliance_display.py` — NEW file. `build_alliance_panel(parent, title, teams, score_variance, *, actual_score=None, win_prob=None, expected_score=None)` and `draw_alliance_chart(parent, red_dist, blue_dist)`, moved verbatim (aside from becoming module-level functions instead of a closure/method) from `ui/match_lookup_tab.py`.

`ui/match_lookup_tab.py` — module docstring updated to mention the refactor; the local `alliance_panel()` closure and `_draw_alliance_chart()` method were removed; `_render()` now imports and calls `build_alliance_panel()` / `draw_alliance_chart()` from `ui/alliance_display.py`, passing `actual_score=(m["red_score"] or 0)/(m["blue_score"] or 0)` exactly as before.

`ui/match_simulator_tab.py` — NEW file. `MatchSimulatorTab(ttk.Frame)`: six digit-validated `ttk.Entry` fields (`self.red_vars/self.blue_vars`, three each) plus a "Simulate" button; `refresh(data)` stores the latest `ComputedData` and re-simulates automatically if all six fields already have values (so a Recalculate/re-import doesn't leave stale numbers on screen); `_simulate()` validates all six fields are filled and distinct, calls `build_alliance_distribution()/compute_match_odds()` exactly as Match Lookup does, and renders via `ui/alliance_display.py`'s two shared functions with `actual_score=None`.

`ui/app.py` — imports and instantiates `MatchSimulatorTab`; adds it to the notebook as a 5th tab labeled "Match Simulator" (after Rankings); `recalculate()` now also calls `self.match_simulator_tab.refresh(self.data)` alongside the other four tabs' refresh calls. No other change to this file.

`README.md` — a Stage 13 section was added (this had been written into the v7 PDF but never actually added to `README.md` in the Stage 13 session — an out-of-sync gap closed in this session), plus a new Stage 14 section documenting this tab, mirroring this PDF section.

Delivered as `frc_scouting_app_16.zip`.

20. Suggested Next Steps (in priority order)

- **Top priority — get a real-machine (real Tkinter window) confirmation of everything delivered across the last two sessions that has only been validated headlessly so far:** specifically, (1) the Match Simulator tab itself (Stage 14, Section 19) — confirm the five tabs display in the right order with "Match Simulator" last, the six entry fields are easy to tab between and clearly labeled Red/Blue, the "Simulate" button and Enter-to-submit both work, the duplicate-team error message displays correctly, and the resulting panels/chart look identical in style to Match Lookup's; and (2) re-confirm, now that Match Lookup's panel/chart code has moved into `ui/alliance_display.py`, that Match Lookup itself still looks and behaves exactly as it did before the refactor (Section 19.4 argues by code review that it should, but an actual visual check is the real test). The user's "latest results look good" comment at the start of this session covers Stages 12 and 13's own deliverables (the Score Distribution chart and the startup-performance changes) but predates Stage 14's work, so Stage 14 specifically still needs its own confirmation.

- Use a second (or the already-tested different) competition's data to make progress on Section 13's bookmarked Google Sheet validation. If that event also has a live Google Sheet with real Match Odds to compare against, run the same kind of spot-check the user did in the Stage 8 session, but this time capture specifics (which match, both systems' win percentages, both systems' expected scores if the sheet exposes them) so a future session has real numbers to root-cause against instead of a qualitative impression. Note that the CSVs uploaded during the Stage 14 session (Untitled_spreadsheet_-_Match_Schedule (1).csv / Untitled_spreadsheet_-_Raw_Data (1).csv, a 45-team/90-match competition) were used only for headless validation in this session and were never actually imported into the app in a UI session — that would be a reasonable next data point to use for this validation item if a live Google Sheet exists for that same event.
- Given the Stage 7 finding that individual-robot 95th-percentile cutoffs run as high as ~589 in the original competition's data (Section 6.6 of the v7 PDF), re-check that figure against whatever range a newly-tested competition's data shows — it's now safe to check this on genuinely separate, non-mixed competition data thanks to the Stage 9 fix.
- Keep an eye on real-world Pick List / Rankings quality across events now that Stage 6 (plain Score Variance) is live in production use, and compare informally against what the live Google Sheet's Pick List would have suggested, if practical. The revert path in Section 10 (originally Section 10.4 of the v7 PDF) is there if this doesn't pan out.
- Consider deferring non-visible tabs' `refresh()` until first shown (see Section 18 / the v7 PDF's Section 18.8) — now five tabs (Match Lookup, Team Lookup, Pick List, Rankings, and the new Match Simulator) are all fully rebuilt at startup before the user can see any but the first. Not implemented in Stage 13 or Stage 14 because it changes GUI construction/event-flow timing that couldn't be verified without a real display.
- Update README.md with any future stage's write-up if it's ever found to be out of sync with this PDF — as of this handoff the two are in sync through Stage 14 (the Stage 13 gap noted above was closed in this session, and README.md's Stage 14 section was written alongside this stage's code changes).
- If the user encounters more Match Schedule typos like the match-47/team-325 case (Section 9) in a new competition's data, the existing `likely_typo_corrections` reporting should surface them automatically on import — but it's worth periodically reviewing the `schedule_conflicts` list too.
- Optional polish ideas, not yet requested: a results/history view comparing Match Odds predictions to actual outcomes over a season (Section 11's 110-match backtest logic would be a good starting point, and the smoothing helper in `ui/alliance_display.py::draw_alliance_chart()` could be reused for any per-match chart such a view might want); a "current competition" label somewhere in the UI so it's visually obvious which event's data is loaded; optionally marking each alliance's actual and/or expected score as a vertical line on the Score Distribution chart; matplotlib's built-in `NavigationToolbar2Tk` (zoom/pan/save) on either chart; and, specific to the new Match Simulator tab, an optional "swap red/blue" button, or a small history list of recently-simulated matchups within the session (not persisted to the database) if the user finds themselves re-typing the same comparisons repeatedly.
- If the "Super Scouting" sheet (Section 2.4) ever resurfaces, its formulas could be added to fill in Team Lookup's team-name / defense-notes / climb-confidence fields.
- If official alliance scores for additional matches become available for an already-loaded competition, re-import an updated Match Schedule CSV using "Update current competition" mode — do NOT use "New competition" mode for this, since that clears all previously-imported raw-data for the event.

21. Data Import Redesign (Stage 4 detail, historical reference)

Unaffected by Stage 14; unchanged from the v7 PDF. This section is preserved as a historical/reference record of the base two-CSV import design as it stood at the end of Stage 4, before the Stage 5 fixes (Section 9) and the Stage 9 import-mode fix (Section 14). It describes: what was removed from the original three-CSV design and why

(Section 21.1); the active Raw Data CSV format, including header-detection and column-mapping rules (21.2); the active Match Schedule CSV format, including its partial-upsert behavior in merge mode (21.3); the zero-collapse bug found and fixed in the Adjusted Score apportionment formula when an official score is still missing (21.4); the final Data menu layout (21.5); and the real-data validation numbers from the original Stage 4 session (21.6). None of Stages 10 through 14 touched the import system at all — refer to the v7 PDF (or the corresponding section of this PDF, unchanged) for the full text of each subsection.

22. How to Resume This Project in a Fresh Session

Provide the following:

- This PDF (project context and every design decision made, through Stage 14).
- `Copy_of_Worlds_Scouting_Sheet_-_FORMULA_EXPORT.csv` — the ground-truth formula export, required for verifying any existing calculation or adding a new one. (Not required to re-verify Stage 13 or Stage 14 specifically — neither changed or added any formula-derived calculation. Still required for any other calculation work.)
- The current `frc_scouting_app/` project folder (or its `.zip` — `frc_scouting_app_16.zip` as of the end of the Stage 14 session, or a newer-numbered zip if one has been shared since). Every `.py` file's docstring is the authoritative spec for what it implements.
- The user's real `Untitled_spreadsheet_-_Raw_Data.csv` and `Untitled_spreadsheet_-_Match_Schedule.csv` files (the original, 75-team competition referenced throughout Sections 9–18), and/or the newer, 45-team/90-match competition's CSVs uploaded during the Stage 14 session (`Untitled_spreadsheet_-_Match_Schedule (1).csv` / `Untitled_spreadsheet_-_Raw_Data (1).csv`), depending on which competition's numbers a future session needs to reproduce.

With those, a fresh session can:

- Re-run the real-data pipeline test to confirm nothing has regressed, using whichever competition's CSVs are relevant, exactly as described in the v7 PDF's Section 21 (originally Section 21) — import both CSVs into a fresh `data/scouting.db`, then call `calculations.pipeline.build_all(my_team_num=1796)`. For the original competition's data, expect 625 imported, 29 skipped, 34 `schedule_conflicts`, 1 `likely_typo_corrections`, a 75-team roster, and a 74-entry Pick List. Stage 14 changes none of these figures — it touched no file under `calculations/`.
- Look up any cell's real formula by filtering the `FORMULA_EXPORT` CSV on the `Sheet` column, exactly as was done throughout Sections 9 and 11's investigations.
- Confirm the Stage 14 changes are present: the notebook should show five tabs in this order — Match Lookup, Team Lookup, Pick List, Rankings, Match Simulator. Match Simulator should show six labeled, digit-only entry fields (three under "Red Alliance:", three under "Blue Alliance:") and a "Simulate" button; submitting a valid, distinct set of six teams should produce the same style of alliance panels and Score Distribution chart Match Lookup shows, but with no "Actual recorded score" line. `ui/alliance_display.py` should exist and export `build_alliance_panel()` and `draw_alliance_chart()`; `ui/match_lookup_tab.py` should import and call those two functions rather than containing its own local panel-building/chart-drawing code. As noted in Section 20's top-priority item, this is the one part of the project that has not yet been confirmed in a real Tkinter window — confirm this early in any resumed session.
- Confirm the Stage 12 and Stage 13 changes are still present exactly as the v7 PDF describes (a "Score Distribution" chart on Match Lookup; `requirements.txt` listing only `matplotlib`; `calculations/distributions.py` with no `scipy` import) — these were reported as "looking good" by the user at the start of the Stage 14 session, which this v8 PDF treats as their real-machine confirmation (Section 17, Section 18).

- Cross-reference this PDF's Section 6 (calculation logic), Sections 9 through 18 (Stage 5 through Stage 13's bug fixes, experiments, and UI additions), and Section 19 (Stage 14's Match Simulator tab) to see exactly which Python function or UI behavior corresponds to which sheet or which prior decision, and why — without needing to re-derive any of the analysis from scratch.

The single most valuable next action for whoever picks this up is the one flagged at the top of Section 20: get a real, running-Tkinter-window confirmation of the Match Simulator tab (Stage 14) and a spot confirmation that Match Lookup's refactored code still looks and behaves identically to before. Everything else delivered through Stage 13 has now been either directly confirmed by the user on their own machine (Stages 10 and 11, and — per the user's "latest results look good" comment at the start of this session — Stages 12 and 13 as well) or is calculation-layer code that Stage 14 did not touch at all.

End of summary. Generated to allow this project to be resumed with no access to the original chat history. This is the v8 revision, updated at the end of the Stage 14 (Match Simulator tab) session to remain accurate and complete through Stage 14.